

CS670: Cryptographic Techniques for Privacy Preservation

Assignment 2 Report

Distributed Point Function (DPF) Generation and Verification

Shubham Rawat (251110070)

Department of Computer Science and Engineering
Indian Institute of Technology, Kanpur

1. Introduction

This report summarizes the implementation of the `gen_queries.cpp` program. The objective was to generate and verify *Distributed Point Functions (DPFs)* — cryptographic primitives that enable two parties to hold shares of a point function $f_{i^*}(x)$ such that:

$$f_{i^*}(i) = \begin{cases} M, & \text{if } i = i^* \\ 0, & \text{otherwise} \end{cases}$$

Each party can locally evaluate its share without revealing the target index or value. The XOR of both evaluations reconstructs the correct output at i^* and zero elsewhere.

2. Implementation Methodology

The program was implemented in **C++17** using **OpenSSL** for cryptographic pseudo-random generation and `std::mt19937_64` for secure randomness.

- `generateDPF()`: Constructs two DPF keys for parties P_0 and P_1 . It uses AES-128 in ECB mode to expand seeds via the PRG (`prg2()`).

The complete process is as follows:

- Convert the target index to its binary representation as a vector.

- Generate two random seeds, set their least significant bits (LSB) to 0 and 1 respectively.
 - Set the flag bits for the two root nodes (0 and 1).
 - Proceed level by level:
 - * For each level, generate children for all nodes.
 - * Compute their XOR values (L_b and R_b).
 - * Using these and the binary vector, compute the correction word (CW) and store it for that level.
 - * Compute flag bits, apply flag correction, and store the corrected flags.
 - * Apply the correction word based on the parent's flag.
 - Finally, compute the final correction word with target value and store it in the CW array.
- `evalDPF()`: Evaluates a DPF key across all indices by expanding seeds level-by-level using the stored correction words and flags.
 - `EvalFull()`: Reconstructs and verifies correctness by XORing both party evaluations, ensuring the target index outputs the desired value while others yield zero.

3. Compilation and Execution

The implementation depends on OpenSSL and standard C++ libraries. To compile and execute:

```
# Install OpenSSL development libraries (for AES)
sudo apt update
sudo apt install -y libssl-dev

# Compile the program linking OpenSSL
g++ gen_queries.cpp -o a -lssl -lcrypto

# Run the program
./a
```

The program will prompt for DPF size and number of DPFs, e.g.:

```
Enter DPF size (e.g 2, 4, 8...) and number of dpfs: 8 1
```

Output: For each DPF, the program displays the chosen target index, target value, and validation results. If reconstruction succeeds, the program prints:

Test Passed